Introduction
Boolean algebra

Boolean Algebra - Танилцуулга

Boolean Algebra (Бүүлийн алгебр) нь:

- Логик утгатай (True/False, 1/0) тоон дээр суурилсан математик систем юм.
- Ихэвчлэн дижитал схем, програмчлал, логик хэлхээ боловсруулахад хэрэглэгддэг.

Хэн зохиосон бэ?

- 1854 онд George Boole хэмээх англи математикчийн гаргасан логик систем.
- Түүний нэрээр Boolean (Бүүл) гэдэг нэршил үүссэн.

Boolean Algebra - Танилцуулга

Яагаад хэрэгтэй болсон бэ?

- Компьютерийн үндсэн ажиллагаа унтарсан/ассан буюу 0 ба 1 дээр тулгуурладаг.
- Эдгээр логик утгуудыг математикаар илэрхийлж, боловсруулж, автомат шийдвэр гаргах боломжийг өгдөг.

Хаана хэрэглэгддэг вэ?

- Компьютерийн схем (Digital Circuit Design)
- Програмчлалын нөхцөл шалгах (if/else logic)
- Автомат удирдлагын систем
- Микропроцессор, электроник дизайн
- Хиймэл оюун ухаан, логик дүгнэлт

Boolean Algebra - Танилцуулга

Boolean Algebra-ийн ач холбогдол

- Логик функцуудыг хялбарчилж бичих боломж.
- Электрон схемийн зардал, хэмжээг бууруулах.
- Илүү хурдан, найдвартай ажиллагаа.

Дүгнэлт

- Boolean Algebra бол орчин үеийн компьютерийн системийн суурь логик.
- Энэ нь логик шийдвэр, автомат систем, электроникийн бүхий л салбарт чухал үүрэгтэй.

Логик схем буюу CPU-ийн гол ажиллах зарчим CPU буюу процессор доторхи бит (0,1) дээр суурилсан логик схемүүд нь Boolean Algebra-г ашиглан:

- Харьцуулах (A > B, A == B)
- Шийдвэр гаргах (if, else)
- Үнэн/худал шалгах (true/false)
- зэрэг үндсэн шийдвэрлэх процессуудыг гүйцэтгэдэг.

Boolean Algebraic Operations гэж юу вэ?

Эдгээр нь логик шийдвэр гаргах үндсэн үйлдлүүд юм.
Бид бодит амьдралд:

- Ямар нэгэн нөхцөл хангасан эсэхийг шалгадаг,

Компьютер ч мөн тийм л зарчмаар ажилладаг.

Boolean Algebra - Танилцуулга

Үндсэн логик үйлдлүүд

Үйлдэл	Тэмдэглэгээ	Тайлбар
AND (болон)	A • B	Зөвхөн хоёулаа 1 бол 1
OR (эсвэл)	A + B	Нэг нь 1 байхад л 1
NOT (эсэргүүцэл)	¬A	A утгыг эсрэг болгоно

Boolean Algebraic Operations гэж юу вэ?

AND үйлдэл (болон / A • B)

- Хэрэглээ: Зөвхөн хоёр нөхцөл зэрэг үнэн (1) бол, шийдвэр гарах.
- Бодит жишээ:
  - Чи “гэрлээ асаах” гэж байна.
  - Гэрэл асаахын тулд түлхүүрийг ON болгох болон цахилгаан байх ёстой.
  - Аль нэг нь байхгүй бол гэрэл асахгүй.

Boolean Algebraic Operations гэж юу вэ?

OR үйлдэл (эсвэл / A + B)

- Хэрэглээ: Аль нэг нөхцөл үнэн (1) байхад шийдвэр гарна.
- Бодит жишээ:
  - Хичээлээс чөлөө авахын тулд:
    - Багш зөвшөөрсөн эсвэл эмчийн бичигтэй байхад болно.
  - Аль нэг нь байхад л хангалттай.

Boolean Algebraic Operations гэж юу вэ?

NOT үйлдэл (эсэргүүцэл / ¬A)

- Хэрэглээ: Нэг утгыг эсрэг болгох.
- Бодит жишээ:
  - Чи "бороо орохгүй" гэж хэлбэл — энэ нь "бороо орно" гэсэн үгийн эсрэг утга.
  - Програм дээр: ¬1 = 0, ¬0 = 1

Boolean Algebraic Operations гэж юу вэ?

🤖 Эдгээр нь хаана хэрэглэгддэг вэ?

- 💻 Компьютерийн процессор (үндсэн логик шийдвэрүүдийг гаргахын тулд)
- 🔌 Цахилгаан хэлхээ, гэрэл асаах схем, тоолуур
- 📱 Апп дотор нөхцөл шалгах (Жишээ: “Хэрвээ хэрэглэгч нэвтэрсэн бол хуудсыг харуул” гэх мэт)
- 🤔 Хиймэл оюун ухаан, логик дүгнэлт гаргах алгоритм

Boolean Algebra-ийн Хуулиуд

Identity Law (Тодорхойлох хууль)

- A + 0 = A
- A • 1 = A

📄 Тайлбар:

Ямар ч утгатай A-г 0-той OR хийхэд A өөрөө хэвээрээ байна.

Ямар ч утгатай A-г 1-тэй AND хийхэд A хэвээрээ байна.

🔸 Жишээ:

- 1 + 0 = 1
- 0 • 1 = 0

Boolean Algebra-ийн Хуулиуд

Idempotent Law (Өөртэйгөө дахин үйлдэх хууль)

- A + A = A
- A • A = A

📄 Тайлбар:

Өөртэйгөө OR эсвэл AND үйлдэл хийхэд A өөрөө л гарна.

🔸 Жишээ:

- 1 + 1 = 1
- 0 • 0 = 0

Boolean Algebra-ийн Хуулиуд

Dominance Law (Ноёрхолын хууль)

- A + 1 = 1
- A • 0 = 0

📄 Тайлбар:

1-тэй OR хийвэл ямар ч A байсан үр дүн 1.

0-той AND хийвэл ямар ч A байсан үр дүн 0.

🔸 Жишээ:

- 0 + 1 = 1
- 1 • 0 = 0

Boolean Algebra-ийн Хуулиуд

Double Negation / Involution Law (Давхар үгүйсгэлийн хууль)

- ¬(¬A) = A

📄 Тайлбар:

A-г хоёр удаа NOT хийхэд A дахин хэвэндээ орно.

🔸 Жишээ:

- A = 1 бол → ¬A = 0 → ¬(¬A) = 1
- A = 0 бол → ¬A = 1 → ¬(¬A) = 0

Boolean Algebra-ийн Хуулиуд

Negation / Complement Law (Эсэргүүцэх хууль)

- A + ¬A = 1
- A • ¬A = 0

📄 Тайлбар:

A ба A-ийн эсрэг утга OR бол үр дүн 1, AND бол 0.

🔸 Жишээ:

- A = 1 → ¬A = 0 → 1 + 0 = 1
- A = 0 → ¬A = 1 → 0 • 1 = 0

Boolean Algebra-ийн Хуулиуд

Commutative Law (Солигдох чанар)

Үйлдлийн дараалал солигдсон ч үр дүн өөрчлөгдөхгүй.

- A + B = B + A
- A • B = B • A

📄 Тайлбар:

A ба B-ийн OR эсвэл AND дараалал солигдох нь хамаагүй — үр дүн ижил.

🔸 Жишээ:

- 1 + 0 = 0 + 1 = 1
- 1 • 0 = 0 • 1 = 0

Boolean Algebra-ийн Хуулиуд

Associative Law (Хам сэдвийн чанар)

Олон утга OR эсвэл AND хийхэд, хаалт (групплэлт)-ийг аль талаас хийх нь хамаагүй.

- (A + B) + C = A + (B + C)
- (A • B) • C = A • (B • C)

📄 Тайлбар:

OR болон AND үйлдэлд ямар дараалаар бүлэглэх нь үр дүнд нөлөөлөхгүй.

🔸 Жишээ:

- (1 + 0) + 1 = 1 + (0 + 1) = 1
- (1 • 1) • 0 = 1 • (1 • 0) = 0

Boolean Algebra-ийн Хуулиуд

Absorption Law (Уусах хууль)

Нэг утга нөгөөгөөр дамжуулан “уусч” илэрхийлэл хялбаршина.

- A + (A • B) = A
- A • (A + B) = A

📄 Тайлбар:

A нь аль хэдийн байна, тэгэхээр A • B эсвэл A + B нэмэлт мэдээлэл өгөхгүй → хасч болно.

🔸 Жишээ:

A + (A • B) = A:

- A = 1, B = 0 → A • B = 0 → A + 0 = 1 = A
- A = 0, B = 1 → A • B = 0 → A + 0 = 0 = A

A • (A + B) = A:

- A = 1, B = 0 → A + B = 1 → A • 1 = 1 = A
- A = 0, B = 1 → A + B = 1 → A • 1 = 0 = A

NOR Gate буюу Universal Logic Gate

NOR Gate гэж юу вэ?
NOR Gate бол OR Gate-ийн NOT хувилбар юм. Энэ нь OR-ийн гаралтын эсрэг утгыг өгдөг.
NOR = NOT (OR)
Товчоор:

- OR-ийн гаралт 1 бол NOR-ийн гаралт 0,
- OR-ийн гаралт 0 бол NOR-ийн гаралт 1.

XOR Gate гэж юу вэ?

XOR gate нь хоёр оролттой (ихэнхдээ), гаралт нь зөвхөн оролтууд өөр байвал 1 гаргана.

- Хоёр оролт адилхан бол гаралт 0
- Хоёр оролт ялгаатай бол гаралт 1

A (оролт)	B (оролт)	A ⊕ B (гаралт)
0	0	0
0	1	1
1	0	1
1	1	0

XNOR (EX-NOR) Gate гэж юу вэ?

XNOR (Exclusive NOR) гэдэг нь XOR (A ⊕ B)-ийн эсрэг (NOT) хувилбар юм.

- 🟡 XNOR = NOT (XOR)

Universal Logic Gates - NAND gate | Boolean Algebra & Logic Gates

NAND Gate гэж юу вэ?
NAND Gate нь AND Gate-ийн NOT хувилбар юм. Өөрөөр хэлбэл, AND-ийн гаралтын эсрэг утга өгдөг.
NAND = NOT (AND)
Товчоор:

- AND-ийн гаралт 1 бол NAND-ийн гаралт 0.
- AND-ийн гаралт 0 бол NAND-ийн гаралт 1.

Sum of Products (SOP) гэж юу вэ?

SOP нь Boolean илэрхийлэл дахь орлуудыг (variables)-ын хооронд AND үйлдлээр хийгдсэн нэгтгэлүүдийг (product) OR үйлдлээр нэмэх (sum) явц юм.

🔑 SOP нь дараах хэлбэрээр илэрхийлэгддэг:

1. AND үйлдлүүдийг нэгтгэнэ.
2. OR үйлдлээр эдгээр нэгтгэлүүдийг нэмнэ.

Жишээ нь:A • B + C • D + E

Product of Sums (POS) гэж юу вэ?

POS гэдэг нь:

- OR үйлдлээр нэгтгэсэн sum terms-уудыг
- AND үйлдлээр үржих (product) хэлбэрийн Boolean илэрхийлэл юм.

Жишээ:

$F = (A + B) \bullet (C + D) \bullet (\neg E + F)$

Minterms (SOP) ба Maxterms (POS)

SOP (Sum of Products) — Үржвэрийн нийлбэр
Тайлбар:

- SOP гэдэг нь "Үржвэрийн нийлбэр" гэсэн утгатай. Энэ нь:
  AND (•) үйлдлүүдийг эхэлж хийдэг (продукт),
  дараа нь OR (+) үйлдлээр нэгтгэдэг (сум буюу нийлбэр).

Жишээ:

$F(A, B, C) = A \bullet \neg B \bullet C + A \bullet B \bullet \neg C$

Энэ нь 2 ширхэг минтерм-ийн нийлбэр.

F = 1 байгаа мөрүүд дээрх хувьсагчийн утгуудын нийлбэр → SOP (Sum of Products)-т хэрэглэгдэнэ.

Maxterms гэж юу вэ? (Product of Sums - POS)

POS (Product of Sums) — Нийлбэрийн үржвэр
Тайлбар:
POS гэдэг нь "Нийлбэрийн үржвэр" гэсэн утгатай. Энэ нь:

- OR (+) үйлдлүүдийг эхэлж хийдэг (сум),
- дараа нь AND (•) үйлдлээр үржүүлдэг (продукт).

Жишээ:
$F(A, B, C) = (A + \neg B + C) \bullet (\neg A + B + C)$
Энэ нь 2 ширхэг макстерм-ийн үржвэр.

F = 0 байгаа мөрүүдийн утгууд →
POS (Product of Sums)-т
хэрэглэгдэнэ.

A	B	C	F
0	0	1	1 → ¬A • ¬B • C
0	1	0	1 → ¬A • B • ¬C
1	0	0	1 → A • ¬B • ¬C
1	1	1	1 → A • B • C

Minterms (SOP) Truth table

A	B	C	F
0	0	0	0 → (A + B + C)
0	1	1	0 → (A + ¬B + ¬C)
1	0	1	0 → (¬A + B + ¬C)
1	1	0	0 → (¬A + ¬B + C)

Maxterms (POS) Truth table

Boolean Expression Simplification

Жишээ 1:
Таны өгсөн илэрхийлэл:
F = (A • B) + (A • C) + (B • C)

- Алхам 1: Хялбаршуулах

Хэрхэн хялбаршуулах вэ?

- (A • B) + (A • C) + (B • C) илэрхийлэлд A ба B хамтран гарах тохиолдол байна.
- Хамгийн эхний алхам бол Distributive Law ашиглах явдал:
  F = A • (B + C) + (B • C)
- Дараа нь оролтыг хялбаршуулахад энэ илэрхийлэл нь хамгийн энгийн хэлбэртэй болсон байна.

- F = A • (B + C) + (B • C)

Distributive Law (Тараах хууль) гэж юу вэ?

Distributive Law нь дараах байдлаар тодорхойлогддог:

1. A • (B + C) = (A • B) + (A • C)
2. A + (B • C) = (A + B) • (A + C)

Тайлбар:

- A • (B + C) гэдэг нь A-г B + C-ийн нийлбэртэй AND үйлдлээр үржүүлэхтэй адил.
- A + (B • C) гэдэг нь A-г B • C-ийн үржвэртэй OR үйлдлээр нийлүүлэхтэй адил.

Энэ хууль нь хаалт ашиглан AND болон OR үйлдлүүдийг хослуулж хялбаршуулах боломжийг олгодог.

Жишээ 1:
Илэрхийлэл:

A•(B+C)

Distributive Law ашиглан:

A•(B+C)=(A•B)+(A•C)

Энд (B + C)-ийг хаалтад авч, A-г тус тусад нь B болон C-ийн нийлбэртэй AND үйлдлээр үржүүлсэн байна.

Karnaugh Map (K-Map) гэж юу вэ?

- K-Map нь Boolean expressions (Бүүлийн илэрхийлэл)-ийг хялбаршуулж, дижитал логик схемүүдийг энгийн болгоход ашиглагддаг хүснэгтэн арга юм.
- Үнэн хүснэгт (Truth Table) дээр үндэслэн утгуудыг бүлэглэж (grouping), хялбар хэлбэрт шилжүүлнэ.

Яагаад хэрэгтэй вэ?

- Илүү товч илэрхийлэл гаргана
- Логик схемийн нарийн төвөгтэй байдлыг багасгана
- Программ хангамж, схем зохиоход илүү хурдан, үр дүнтэй болгодог

Жишээ 1: 2 хувьсагчтай K-Map

Үнэн хүснэгт (Truth Table)

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

K-Map хүснэгт

A \ B	0	1
0	0	1
1	1	1

Зорилго:
“K-map Numerical with 3 Variables – SOP Boolean Expression” буюу:
3 хувьсагчтай K-map ашиглан,
SOP хэлбэрийн логик илэрхийлэл гаргах

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

AB \ C	0	1
0	0	1
1	1	1
10	0	1
11	1	1

Алхам 3: 1-үүд дээр үндэслэн SOP гаргах
1 байгаа бүх байршлыг бичье:

- A=0 B=0 C=1 → A'B'C
- A=0 B=1 C=0 → A'BC'
- A=0 B=1 C=1 → A'BC
- A=1 B=0 C=1 → AB'C
- A=1 B=1 C=0 → ABC'
- A=1 B=1 C=1 → ABC

AB \ C	0	1
0	0	1
1	1	1
10	0	1
11	1	1

Алхам 4: SOP (Sum of Products) хэлбэр:
Бүх 1 байршлын хувьд AND илэрхийлэл, дараа нь бүгдийг OR хийнэ.

F=A′B′C+A′BC′+A′BC+AB′C+ABC′+ABC

Variable K-Map гэж юу вэ?

4 хувьсагчтай Бүүлийн илэрхийллийг (Boolean expression) хамгийн хялбар хэлбэрт шилжүүлэхэд 4x4 хэмжээтэй K-Map ашигладаг.

AB \ CD	0	1	11	10
0	1	1	1	0
1	0	1	1	1
11	0	1	1	1
10	0	1	1	1

A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

6 ширхэг 1

Энэ бол 4 хувьсагчтай K-map (Karnaugh Map) бөгөөд дараах байдлаар өгөгджээ:

AB \ CD	0	1	11	10
0	1	1	1	0
1	0	1	1	1
11	0	1	1	1
10	0	1	1	1

Алхам 1: K-map дээр 1-үүдийг ажиглах
K-map дээр нийт 6 ширхэг “1” дараалж байрласан хэсгийг том багц болгоно.
Доорх нүдүүдэд 1 байна:
00\00 ✅
00\01 ✅
00\11 ✅
01\01 ✅
01\11 ✅
01\10 ✅
11\01 ✅
11\11 ✅
11\10 ✅
10\01 ✅
10\11 ✅
10\10 ✅
→ Эндээс доод талын 3 мөрийн баруун 3 багана (01, 11, 10) хэсгийг 6 ширхэг "1" багц болж байна.

Алхам 2: Том бүлэг сонгох

Бидэнд 6 ширхэг дараалсан 1 байгаа бүс:

01\01, 01\11, 01\10
11\01, 11\11, 11\10
10\01, 10\11, 10\10

→ Эдгээр 1-үүд нь 3 мөр × 3 багана мэт харагдаж байгаа боловч K-map зөв дүрэм нь 2^n хэмжээтэй багц (1, 2, 4, 8, 16 гэх мэт) байх ёстой.
Тэгэхээр бид дараах 6 ширхэг “1”-ийг нэг багц болгож болно:

AB \ CD	0	1	11	10
0	1	1	1	0
1	0	1	1	1
11	0	1	1	1
10	0	1	1	1

AB \ CD	1	11
1	1	1
11	1	1
10	1	1

Алхам 3: Илэрхийлэл гаргах

Энэ багц нь дараах AB ба CD утгууд дээр “1” агуулж байна:

- B хувьсагчийн 0 болон 1 утгууд тус тус 01, 10, 11 зэрэг байршилд байгаа тул B нь өөрчлөгдсөн байна.

- CD-ийн 2-р бит 01 болон 11 байрлаж байгаа тул бид D = 1 авч болно
- Харин C хувьсагч нь 01 болон 11 үсрэн орж байгаа учраас C өөрчлөгдөнө.

Хамгийн чухал хэсэг:

- D = 1 гэсэн утга нь CD = 01 болон CD = 11 гэсэн утгуудтай байдаг.

AB \ CD	1	11
1	1	1
11	1	1
10	1	1

Эцсийн хариу:
Том 6 ширхэг “1” бүхий багцын илэрхийлэл бол:
F = D

Хувьсагчтай K-map ашиглан SOP (Sum of Products)

AB \ CD	0	1	11	10
0	1	1	1	0
1	1	0	0	0
11	0	0	1	0
10	0	0	0	0

Боломжит 1-үүдийг бүлэглэнэ:

- Бүлэг 1: (00,00) ба (00,01) → A'=1, B'=1, C'=1 → A'B'C'
- Бүлэг 2: (00,01) ба (00,11) → A'=1, B'=1, D=1 → A'B'D
- Бүлэг 3: (01,00) → A=0, B=1, C=0, D=0 → A'BC'D'
- Бүлэг 4: (11,11) → A=1, B=1, C=1, D=1 → ABCD

Эцсийн SOP илэрхийлэл

F = A'B'C' + A'B'D + A'BC'D' + ABCD

4-Хувьсагчтай K-Map – “Don’t Care” нөхцөлтэй SOP Илэрхийлэл

📌 Don't Care гэж юу вэ?

- Don't Care Condition (Тооцоонд оролцуулах эсэх нь хамаагүй нөхцөл) гэдэг нь:
  - Гарц (Output) нь “0” ч бай “1” ч бай алдаа үүсгэхгүй.
  - Ихэвчлэн нийлмэл систем дээр зарим оролтын хувилбар ашиглагддаггүй, эсвэл ачаалалгүй байдаг.
  - Тиймээс эдгээр нөхцлийг 1 болгох эсвэл үл тоох боломжтой.
  - Энэ нь хялбаршуулалт (simplification) хийхэд тусалдаг.

K-Map хүснэгт (4 хувьсагч)

A, B, C, D хувьсагчтай 4-variable K-map (SOP хэлбэр)

Доорх K-map дээр:

- 1 бол F = 1 гэсэн утгатай.
- X бол Don't Care нөхцөл.

Grouping хийж хялбаршуулах (SOP)
Don't care (X) утгуудыг ашиглан том багц үүсгэнэ.

📌 Бүлэг 1:

- (11,11), (11,10), (10,11), (10,10) → 4 “1”
- Энд A = 1, D = 1 ➜ AD

📌 Бүлэг 2:

- (00,00), (00,01), (01,01), (01,11) → 2 “1” + 2 “X”
- A = 0, B = *, C = 1 ➜ A'C

📌 Бүлэг 3:

- (11,00), (11,01) → 2 “1”
- A = 1, B = 1, C = 0 ➜ AB·C'

AB \ CD	0	1	11	10
0	1	1	x	0
1	1	x	1	0
11	0	1	1	1
10	0	0	1	1

Эцсийн хариу (SOP хэлбэрээр):

F = AD + A'C + ABC'

✨ Давуу тал:

- Don't Care нөхцлийг ашигласнаар:
  - Багцлал томорно
  - Илэрхийлэл богиносно
  - Схем хялбар болно

K-map Problem with 3 Variable POS Boolean Expresson

За окей! Энд 3 хувьсагчтай K-map бодлого өгөгдөж байгаа бөгөөд POS (Product of Sums – нийлбэрийн үржвэр) хэлбэрээр илэрхийлэл гаргахыг хүсч байна. POS нь 0-ууд дээр анхаардаг гэдгийг санах хэрэгтэй.

A\BC	0	1	11	10
0	1	1	0	1
1	0	0	0	1

Алхам 1: 0-уудын байршлыг тодорхойлъё

0 байна дараах байршлуудад:

- (0,11)
- (1,00)
- (1,01)
- (1,11)

Эдгээр 0-уудыг томоор бүлэглэе (POS учраас 0-уудыг бүлэглэнэ)

📌 Бүлэг 1:

(1,00), (1,01) → A = 1, B = 0 ➜ (A)(B')

POS бол эсрэгээр нь бичнэ → (A + B)

📌 Бүлэг 2:

(1,11), (0,11) → C = 1 ➜ (C)

POS хэлбэр: A=*, B=1, C=1 → (B)(C) → (B + C)

POS илэрхийлэл

F = (A + B)(B + C)

✳️ POS-той холбоотой тайлбар:

- SOP → 1-үүд дээр ажилладаг, AND-уудын OR
- POS → 0-ууд дээр ажилладаг, OR-уудын AND

Half Adder – Combinational Circuit

Half Adder гэж юу вэ?

Half Adder нь 2 битийн binary тоог нэмэх зориулалттай combinational circuit юм.
Түүний үүрэг бол:

- 2 оролт (Input): A, B
- 2 гаралт (Output):
  - Sum (S) – нийлбэр
  - Carry (C) – дамжигч утга

Truth Table (Үнэн хүснэгт):

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions:

- Sum = A ⊕ B → (XOR)
- Carry = A • B → (AND)

Тайлбар:

- XOR gate (⊕): A, B 2 өөр байвал 1, ижил байвал 0 → Sum
- AND gate (•): A ба B хоёулаа 1 байвал → Carry = 1

Full Adder гэж юу вэ?

Full Adder нь гурван оролттой:

- A, B → нэмэх гэж байгаа хоёр бит
- Cin → өмнөх оронтын Carry (ачааны утга)

Хоёр гаралттай:

- Sum → энэ оронтын нийлбэр
- Cout → дараагийн оронт руу шилжих Carry

Boolean Илэрхийлэл

1. Sum = A ⊕ B ⊕ Cin
   - Энэ нь XOR хаалганы цуврал ашигладаг.
   - Өөрөөр хэлбэл, A ба B-гийн Half Adder-ийн Sum-г дахин XOR хийх.
2. Carry (Cout) = AB + BCin + ACin
   - Энэ нь 3 ширхэг AND, 2 ширхэг OR хаалгаар хийгдэнэ.

Давуу тал

- Full Adder-ийг ашиглан хэдэн ч битийн тоо нэмэх боломжтой.
- N-битийн нэмэгч хийхийн тулд олон Full Adder-ийг цуваа холбоно.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Энгийн жишээ

A = 1, B = 1, Cin = 1

- Sum = 1 ⊕ 1 ⊕ 1 = 1
- Cout = (1•1) + (1•1) + (1•1) = 1 + 1 + 1 = 1 (зөвхөн 1 гарна, логик OR хийж байгаа тул)

Multiplexer гэж юу вэ?

Multiplexer буюу Олон-орох нэг-гаралт төхөөрөмж нь:

- Олон оролтоос нэгийг нь сонгоод гаралт руу дамжуулдаг комбинацийн логик схем юм.
- 4:1 MUX нь 4 оролттой (I_0, I_1, I_2, I_3) бөгөөд 2 ширхэг сонгогч оролт (S_0, S_1)-оор аль input-ийг гаралт (Y) болгож өгөхийг шийднэ.

Ажиллах зарчим:

S_1	S_0	Y =
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Сонгогч оролт ямар утгатай байгаагаас шалтгаалаад, тэр оролт гаралт болж гардаг.

Boolean Илэрхийлэл:

4:1 MUX-н гаралт Y нь дараах байдлаар илэрхийлэгдэнэ:

$$Y = \overline{S1} \cdot \overline{S0} \cdot I0 + \overline{S1} \cdot S0 \cdot I1 + S1 \cdot \overline{S0} \cdot I2 + S1 \cdot S0 \cdot I3$$

Логик хаалгаар хэрэгжүүлэх:
Танд хэрэгтэй:

- NOT (inverter) → S_1 болон S_0-г үгүйсгэх
- AND хаалга → тус бүрийн нөхцөл шалгах
- OR хаалга → 4 нөхцлийн нийлбэр = Y

Хэрэглээ:

- Шийдвэр гаргах логик
- Өгөгдөл сонголт (data selector)
- Микропроцессор дахь хаягийн чиглүүлэлт
- Логик функцүүдийг хэрэгжүүлэхэд

Товч ойлголт:

Multiplexer нь:

- Гаралтдаа нэг л оролт дамжуулдаг
- Сонгогч оролтоор аль input-г гаргах вэ гэдгийг шийддэг
- Boolean илэрхийлэл нь сонгогч бүрийн AND нөхцөл + OR нийлбэртэй байна

1:4 Demultiplexer гэж юу вэ?

DEMUX буюу Demultiplexer нь 1 оролтыг олон гаралтад зориулалтын дагуу салгаж өгдөг combinational logic circuit юм.

1:4 DEMUX:

- 1 оролт
- 4 гаралт
- 2 selector input (S1, S0) — аль гаралтад дамжуулахыг шийддэг

🧠 Ашиглах тохиолдол:

- Сигналыг нэг зэрэг олон төхөөрөмж рүү хяналттай тараах
- Data routing, address decoding зэрэгт өргөн ашиглагддаг

Үнэн хүснэгт (Truth Table):

S1	S0	Y0	Y1	Y2	Y3
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

I = Оролтын утга. Зөвхөн сонгогдсон гаралтаар л дамжин гардаг.

Boolean Илэрхийлэл:

- Y0 = I · S1′ · S0′
- Y1 = I · S1′ · S0
- Y2 = I · S1 · S0′
- Y3 = I · S1 · S0

(′ гэдэг нь NOT буюу үгүйсгэл)

Encoder гэж юу вэ?

Encoder нь нэгээс олон оролт руу нэг гаралт дамжуулах комбинацийн логик схем юм. Энэ схем нь олон оролтыг энкод хийж, бинар кодоор гаралт өгдөг.

4x2 Encoder (4 оролт, 2 гаралт)

Илэрхийлэл:

- $A_1 = I_3 + I_2$
- $A_0 = I_1 + I_0$

I_3	I_2	I_1	I_0	A_1	A_0
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	1	0
0	0	1	1	1	1
0	1	0	0	1	0
0	1	0	1	1	1
1	0	0	0	1	0
1	0	0	1	1	1

Decoder гэж юу вэ?

Decoder нь бинар кодоос олон гаралт руу дамжуулдаг комбинацийн логик схем юм. Энэхүү схем нь бинар тоог олон гаралт болгож хөрвүүлдэг.

Encoder болон Decoder-ийн хэрэглээ:

- Encoder: Үндсэндээ олон оролтыг бинар код болгон хувиргахад ашиглагддаг. Жишээ нь, системийн хаягууд.
- Decoder: Бинар кодоос гаргалт (output) хийж, бусад үйлдлийг тогтмол хянахад хэрэглэгддэг.

2x4 Decoder (2 оролт, 4 гаралт)

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Илэрхийлэл:

- $Y_3 = S_1 \cdot S_0'$
- $Y_2 = S_1' \cdot S_0$
- $Y_1 = S_1 \cdot S_0$
- $Y_0 = S_1' \cdot S_0'$

Priority Encoder гэж юу вэ?

Priority Encoder нь олон оролттой схем бөгөөд олон оролтоос хамгийн их priority-тай оролтыг сонгоно. Оролт тус бүрийн priority тодорхой байдаг:

- Үндсэн ажиллах зарчим: Хэрэв хэд хэдэн оролт 1 утгатай байвал хамгийн өндөр priority-тай оролт нь гаралтад тусгагдана.

🔹 Жишээ:

- 4:2 Priority Encoder
  - Оролт: I_3, I_2, I_1, I_0 (I_3 нь хамгийн өндөр priority)
  - Гаралт: 2 битийн output (A_1, A_0) болон valid (V) сигналыг өгнө.

I_3	I_2	I_1	I_0	A_1	A_0	V
0	0	0	0	0	0	0
0	0	0	1	0	1	1
0	0	1	0	1	0	1
0	0	1	1	1	0	1
0	1	0	0	1	1	1
0	1	0	1	1	1	1
0	1	1	0	1	1	1
0	1	1	1	1	1	1
1	0	0	0	1	0	1
1	0	0	1	1	0	1
1	0	1	0	1	0	1
1	0	1	1	1	0	1
1	1	0	0	1	1	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

Boolean илэрхийлэл:

A_1 (Output for bit 1):

$$A_1 = I_3 + I_2$$

A_0 (Output for bit 0):

$$A_0 = I_3 + I_1$$

Valid (V):

$$V = I_3 + I_2 + I_1 + I_0$$

I_3	I_2	I_1	I_0	A_1
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

$$A_1 = I_3 + I_2$$

- Учир нь зөвхөн $I_3 = 1$ эсвэл $I_2 = 1$ үед A_1 утга нь 1 болж байна.

I_3	I_2	I_1	I_0	A_0
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

$$A_0 = I_3 + I_1$$

Учир нь зөвхөн $I_3 = 1$ эсвэл $I_1 = 1$ үед A_0 утга нь 1 болж байна.

✅ Valid гаралтын утга:

$Valid = I_3 + I_2 + I_1 + I_0$

🟨 Учир нь оролтын аль нэг нь 1 болсон тохиолдолд Valid нь 1 болдог. Харин бүгд 0 байвал Valid = 0.

📌 Эцэст нь:

- $A_1 = I_3 + I_2$
- $A_0 = I_3 + I_1$
- $V = I_3 + I_2 + I_1 + I_0$

Decimal to BCD Encoder гэж юу вэ?

- Энэ нь 10 оролттой (D_0–D_9) бөгөөд зөвхөн нэг нь тухайн үед 1 байдаг.
- Гаралт нь 4 битийн BCD код (A_3, A_2, A_1, A_0) байна.
- Энэхүү encoder нь зөвхөн 0–9 тоог хөрвүүлж өгдөг.

Decimal Input	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_3	A_2	A_1	A_0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	0	0	1	0	0	0	0	1
2	0	0	0	0	0	0	0	1	0	0	0	0	1	0
3	0	0	0	0	0	0	1	0	0	0	0	0	1	1
4	0	0	0	0	0	1	0	0	0	0	0	1	0	0
5	0	0	0	0	1	0	0	0	0	0	0	1	0	1
6	0	0	0	1	0	0	0	0	0	0	0	1	1	0
7	0	0	1	0	0	0	0	0	0	0	0	1	1	1
8	0	1	0	0	0	0	0	0	0	0	1	0	0	0
9	1	0	0	0	0	0	0	0	0	0	1	0	0	1

Boolean Илэрхийлэл (A_3, A_2, A_1, A_0)

- $A_3 = D_8 + D_9$
- $A_2 = D_4 + D_5 + D_6 + D_7$
- $A_1 = D_2 + D_3 + D_6 + D_7$
- $A_0 = D_1 + D_3 + D_5 + D_7 + D_9$

Эдгээрийг OR (эсвэл) логик ашиглан хэрэгжүүлнэ.

Схем (Circuit Diagram)
Ингэж холбогдоно:

- 10 ширхэг оролт бүхий D_0-с D_9 гартай.
- Дээрх Boolean илэрхийллүүдийг хэрэгжүүлж 4 ширхэг гаралт гарна (A_3-с A_0).

Хэрэглээ:

- Калькулятор
- Тоолуур (counter)
- Keypad товчлуур шифрлэлт
- Digital системийн оруулгын нэгийг олон болгож хөрвүүлэх

Hexadecimal to Binary Encoder

Encoder гэж юу вэ?

Encoder (кодлогч) бол олон оролттой бөгөөд зөвхөн нэг оролт идэвхтэй байх үед түүнийг бинар хэлбэрээр илэрхийлдэг комбинацийн схем юм.

Hexadecimal to Binary Encoder гэж юу вэ?

Hexadecimal to Binary Encoder нь 16 оролттой бөгөөд 4 битийн бинар код (Binary) болгон хувиргана. Энэ нь 0–F (16 оронтой) hexadecimal утгуудыг 4 битийн binary болгоно.

Үндсэн ойлголт:

- Оролтууд: D_0, D_1, D_2, ..., D_{15} (16 ширхэг оролт)
- Гаралтууд: A_3, A_2, A_1, A_0 (4 битийн binary код)
- Зөвхөн нэг оролт 1, бусад нь 0 байх ёстой.

Decimal (Hex Input)	D_0–D_{15}	A_3	A_2	A_1	A_0
0 (D_0 = 1)	0000 0000 0000 0001	0	0	0	0
1 (D_1 = 1)	0000 0000 0000 0010	0	0	0	1
2	…	0	0	1	0
3	…	0	0	1	1
4	…	0	1	0	0
5	…	0	1	0	1
6	…	0	1	1	0
7	…	0	1	1	1
8	…	1	0	0	0
9	…	1	0	0	1
A (10)	…	1	0	1	0
B (11)	…	1	0	1	1
C (12)	…	1	1	0	0
D (13)	…	1	1	0	1
E (14)	…	1	1	1	0
F (15)	…	1	1	1	1

Boolean илэрхийлэл (гаргалгаануудын хувьд)

- $A_0 = D_1 + D_3 + D_5 + D_7 + D_9 + D_{11} + D_{13} + D_{15}$
- $A_1 = D_2 + D_3 + D_6 + D_7 + D_{10} + D_{11} + D_{14} + D_{15}$
- $A_2 = D_4 + D_5 + D_6 + D_7 + D_{12} + D_{13} + D_{14} + D_{15}$
- $A_3 = D_8 + D_9 + D_{10} + D_{11} + D_{12} + D_{13} + D_{14} + D_{15}$

Илүү энгийн болгохын тулд K-map ашиглан эдгээр илэрхийллийг хялбарчилж болно.

K-map-ийн бүтэц:

A_3, A_2, A_1, A_0-ийн K-map-ыг 4 оролтын утгуудыг зохиох замаар нэг нэгээр байрлуулах болно.

A_3-ийн K-map:

D_0–D_{15}	0	1	11	10
0	0	0	0	1
1	0	0	1	1
11	1	1	1	1
10	1	1	1	1

K-map-ын үндсэн үр дүн:

- $A_3 = D_8 + D_9 + D_{10} + D_{11} + D_{12} + D_{13} + D_{14} + D_{15}$
- K-map-д A_3-ийг 8 ширхэг 1 бүхий том багцаар илэрхийлж болох бөгөөд энэ нь D_8–D_{15} бүхий багц юм.

A_2-ийн K-map:

D_0–D_{15}	0	1	11	10
0	0	0	1	1
1	0	1	1	1
11	0	1	1	1
10	1	1	1	1

K-map-ын үндсэн үр дүн:

- $A_2 = D_4 + D_5 + D_6 + D_7 + D_{12} + D_{13} + D_{14} + D_{15}$
- K-map-д A_2-ийг 8 ширхэг 1 бүхий том багцаар илэрхийлж болох бөгөөд энэ нь D_4–D_7 болон D_{12}–D_{15} бүхий багцууд юм.

A_1-ийн K-map:

D_0–D_{15}	0	1	11	10
0	0	1	1	1
1	0	1	1	1
11	1	1	1	1
10	1	1	1	1

K-map-ын үндсэн үр дүн:

- $A_1 = D_2 + D_3 + D_6 + D_7 + D_{10} + D_{11} + D_{14} + D_{15}$
- K-map-д A_1-ийг 8 ширхэг 1 бүхий том багцаар илэрхийлж болох бөгөөд энэ нь D_2–D_3 болон D_6–D_7 болон D_{10}–D_{11} бүхий багцууд юм.

A_0-ийн K-map:

D_0–D_{15}	0	1	11	10
0	1	1	1	1
1	1	1	1	1
11	1	1	1	1
10	1	1	1	1

K-map-ын үндсэн үр дүн:

- $A_0 = D_1 + D_3 + D_5 + D_7 + D_9 + D_{11} + D_{13} + D_{15}$
- K-map-д A_0-ийг 8 ширхэг 1 бүхий том багцаар илэрхийлж болох бөгөөд энэ нь D_1–D_3 болон D_5–D_7 болон D_9–D_{11} болон D_{13}–D_{15} бүхий багцууд юм.

Эцсийн Boolean илэрхийлэл:

- $A_3 = D_8 + D_9 + D_{10} + D_{11} + D_{12} + D_{13} + D_{14} + D_{15}$
- $A_2 = D_4 + D_5 + D_6 + D_7 + D_{12} + D_{13} + D_{14} + D_{15}$
- $A_1 = D_2 + D_3 + D_6 + D_7 + D_{10} + D_{11} + D_{14} + D_{15}$
- $A_0 = D_1 + D_3 + D_5 + D_7 + D_9 + D_{11} + D_{13} + D_{15}$

Хэрэглээ:

- Hexadecimal to Binary Encoder схем нь олон тооны оролтод үйлдэл гүйцэтгэх үед бинар код ашиглахад шаардлагатай.

Хэрэглээ:

- **Микропроцессор: Нех дохиог binary болгон хувиргахад**
- **Цифрэн дэлгэц: Баганууд ыг сонгох**
- **Мэдээллийн кодчлол: Оролтыг binary кодлох**